

Lecture 2: Curry-Howard Correspondence

Recall: A proposition: $\forall x, \forall y : \underbrace{P(x, y)}_{\text{Predicates}}$

Upshot: Propositions are precise statement formulated in the context of a theory.

Formal proof: is a sequence of arguments where each step is

- 1) Axiom
- 2) Known Proposition

replace eq. terms
 $x=y$

- 3) Application of a rule of inference - $\frac{P \wedge (P \rightarrow Q)}{Q}$

Lean { 1) Programming language that enables us to define propositions & logical statements

{ 2) Verify the correctness of a formal proof.

Today: How does that work?

Answer: Type theory.

"Usual" mathematics: is based on set theory

Proving a proposition works as follows. in a theory T .

- 1) Pick set M (model) that satisfies all the axioms of our theory
- 2) Every $x \in M$ satisfies the proposition.

Problem: Computers don't like sets!

However, Computers like rules!

Type theory is an alternative foundation for math, it is based on rules (instead of sets).

Ex. $\{p \in \mathbb{N} \mid \boxed{p \text{ is Prime}}\} = P \subset \mathbb{N}$

A computer cannot solve the set but it can

verfy if a nat. number is prime.

	Set theory	Type theory
object	x (abstract) object lives in the universe	x term (sequence symbols)
collections	set M	type T , "set" of rules to decide if a term belongs to T
judgements	$x \in M$	$x : T$
operations	$M_1 \cup M_2$ (unions) $M_1 \times M_2$ (products) $= \{(x, y) \mid x \in M_1, y \in M_2\}$	$T_1 \sqcup T_2$ (sum type) $x : T_1 \sqcup T_2$ (x satisfies sub T_1 or T_2) $T_1 \times T_2$ (product type) $z : T_1 \times T_2$ if $z = (x, y)$ where $x : T_1$ $y : T_2$

Ex Rational numbers as a set

$$\mathbb{Q} = \left\{ x \in \mathbb{R} \mid \exists n, m \in \mathbb{Z} \quad n \neq 0 \quad x = \frac{n}{m} \right\} \quad \text{periodic} \Leftrightarrow \text{repeating.}$$

$$\mathbb{Q} = \{ x \in \mathbb{R} \mid x \text{ periodic decimal expansion} \}$$

$$\mathbb{Q} = \{ x \in \mathbb{R} \mid \exists n, m \in \mathbb{Z}, \text{ coprime}, x = \frac{n}{m} \}$$

All of the above are literally the same set!

Ex. Real numbers a type. I get 3 different types.

Q_1 : expressions of the $\frac{n}{m}$, $n, m: \mathbb{Z}$, $m \neq 0$.

$a_i = 0, 1, 2, 3, 4, 5, \dots$

Q_2 : expression $a_n a_{n-1} \dots a_0$, $a_0, a_1, a_2, \dots$ where the

Q_3 : expressions of the form $\frac{n}{m}$, $n, m: \mathbb{Z}$, $m \neq 0$, n, m are coprime.
 ~~Seq. $a_{-1}, a_{-2}, \dots$ become periodic.~~

These are not the same type, so

when we talk about a real number in type theory we have specify the version we're using.

Objection: 1) Q_1 ; $\frac{n}{m}$ is not a unique expression.
 ^{shouldn't}

Solution: Type theory allows us to specify when $x = y$ for $x: T$ & $y: T$.

$$\frac{n}{m} = \frac{n'}{m'} \iff n \cdot m' = n' \cdot m.$$

Abstract examples

1) Function type: Given types T_1, T_2 there is a new type $T_1 \rightarrow T_2$.

Functions are rules of translation.

2) Empty type: $\emptyset$ or $\perp$, is defined as the type for which no term satisfies $x: \emptyset$.

3) The unit type: Unit, T

Has exact one term that satisfies $x: \text{Unit}$,
we write: $()$ as unit, $*$

Ex function type: $f: (\mathbb{Z} \rightarrow \mathbb{Q}_1)$

f is the rule $n: \mathbb{Z} \rightarrow \frac{n}{1}: \mathbb{Q}_1$.

Unit type: $\overset{\uparrow}{()} : \text{Unit}$

What is a term of type $(\text{Unit} \rightarrow \mathbb{Z})$?

$f: \text{Unit} \rightarrow \mathbb{Z}$ is given by specifying exactly one integer.

Curry - Howard correspondence

Fix a theory T .

The idea behind the correspondence is:

- 1) There exist a type $\tilde{T}$ that corresponds to the theory T . (variables in logic now live in the type $\tilde{T}$).
- 2) For every proposition φ we can find a type $T(\varphi)$ st. φ provable $\Leftrightarrow$ there exists a term of type $T(\varphi)$.

Ex. Easiest propositions are

- $\varphi = \text{True}$ (proof is trivial)

the corresponding type $T(\text{True}) = \text{Unit}$.

- $\varphi = \text{False}$ (by def. has no proof)

$$\rightarrow T(\text{False}) = \emptyset \quad (\text{Empty type}).$$

* φ_1 & φ_2 are two propositions.

Which type corresponds: $T(\varphi_1 \wedge \varphi_2) = ?$

$\therefore T(\varphi_1 \vee \varphi_2) = ?$

1) $T(\varphi_1 \wedge \varphi_2)$ is supposed to have a term

satisfying $p: T(\varphi_1 \wedge \varphi_2) \leftrightarrow \varphi_1 \wedge \varphi_2$ is provable

$\leftrightarrow \begin{cases} \varphi_1 \text{ is provable} \& \\ \varphi_2 \text{ is provable.} \end{cases}$

$\begin{cases} p_1: T(\varphi_1) \text{ and} \\ p_2: T(\varphi_2) \end{cases} \leftrightarrow$

$\leftrightarrow$

$(p_1, p_2): T(\varphi_1) \times T(\varphi_2).$

Exc. $T(\varphi_1 \vee \varphi_2) = T(\varphi_1) \sqcup T(\varphi_2)$.

Finally: What is the type $T(\varphi_1 \Rightarrow \varphi_2)$?

A: The function type $(T(\varphi_1) \rightarrow T(\varphi_2)).$

A proof of $\varphi_1 \Rightarrow \varphi_2$ is essentially showing

"Given a proof of φ_1 , we can prove φ_2 ".

Ex. Remember the defn. of $\neg \varphi := \varphi \Rightarrow \text{False}$.

Exc. $T(\neg \varphi_1 \vee \varphi_2) = ?$

Summary: 1) The Lean theorem is a program that

"constructs" the type $T(\varphi)$ given a proposition φ .

2) When write a proof in Lean, the theorem prover

checks that the resulting term is of type $T(\varphi)$.
